docker_lab

Guia de instalação e uso — execução de experimentos com CPU, RAM e GPU controlados em servidor compartilhado

Projeto: github.com/lucas-mendonca-andrade/docker_lab

1. Para que serve

Vários alunos usam a mesma máquina de laboratório. Sem controle, um único experimento pesado podia esgotar CPU, memória ou GPU, derrubar o acesso remoto de todos e até reiniciar a máquina. O docker_lab resolve isso com três mecanismos:

Execução em container
Todo script Python roda dentro de um container Docker, com a quantidade de CPU, RAM e VRAM que o próprio aluno pediu, e nada além disso.

Teto da máquina
A soma do que todos os jobs reservaram nunca passa de 80% dos cores e da VRAM, nem de 95% da RAM. Um pedido que ultrapassaria o teto é recusado na hora.

Proteções na máquina
A potência da GPU fica limitada a 80%, e rodar `python3` direto no terminal mostra um aviso em vez de executar.

Como uma execução acontece

job.env o aluno descreve o script e os recursos → **docker-lab run** valida a configuração e monta a imagem → **Reserva** confere o teto da máquina somando todos os jobs → **Container** roda em segundo plano, preso aos limites pedidos → **Fim** libera a reserva e registra no histórico

Recurso	Limite por job (definido no job.env)	Teto conjunto (todos os jobs)
CPU	`CORES`: quantidade de cores	80% dos cores da máquina
RAM	`MEMORY_GB`: passou disso, o job é encerrado	95% da RAM
VRAM (memória da GPU)	`GPU_MEMORY_GB`: passou disso, o job recebe `CUDA out of memory`	80% da VRAM de cada GPU
Potência da GPU	não é definida por job	80% do limite atual da GPU (TITAN V: 250 W → 200 W)

O processamento da GPU não é reservado por job: quando dois jobs usam a mesma GPU, o driver da NVIDIA alterna entre eles, e cada um fica proporcionalmente mais lento.

2. Instalação (administrador, uma única vez)

A instalação é feita na máquina do laboratório por quem tem `sudo`. São dois comandos:

```
sudo git clone https://github.com/lucas-mendonca-andrade/docker_lab.git /opt/docker_lab
sudo /opt/docker_lab/setup.sh
```

> **Instale em `/opt/docker_lab`, como root, e não mova a pasta depois.** Se a pasta ficar na home de um aluno, os outros usuários não conseguem acessá-la, e esse aluno poderia alterar scripts que rodam como root. Para mudar de lugar: `undo.sh`, mover a pasta e rodar `setup.sh` de novo.

O `setup.sh` faz seis coisas, e pode ser rodado de novo sem problema (por exemplo, depois de um `git pull`):

Passo	O que faz
1. Instalação compartilhada	Libera `state/` e `logs/` para todos os usuários, que passam a gravar no mesmo controle de reservas. Usuários criados no futuro já funcionam, sem nenhum comando extra.
2. Potência da GPU	Limita cada GPU a 80% do limite de potência atual e instala um serviço do systemd que reaplica o limite a cada boot.
3. Comando `docker-lab`	Instala o comando `docker-lab` para todos os usuários.
4. Bloqueio de Python direto	Faz `python3 script.py` no terminal mostrar um aviso em vez de executar, inclusive com venv ativado. Os binários do sistema não são alterados.
5. Acesso ao Docker	Coloca todos os usuários da máquina no grupo `docker`, inclusive os criados depois.
6. Fila de execução	Instala o escalonador (serviço `docker_lab-scheduler`), que inicia os experimentos da fila e envia os e-mails, e cria `/etc/docker_lab/email.env` para as credenciais.

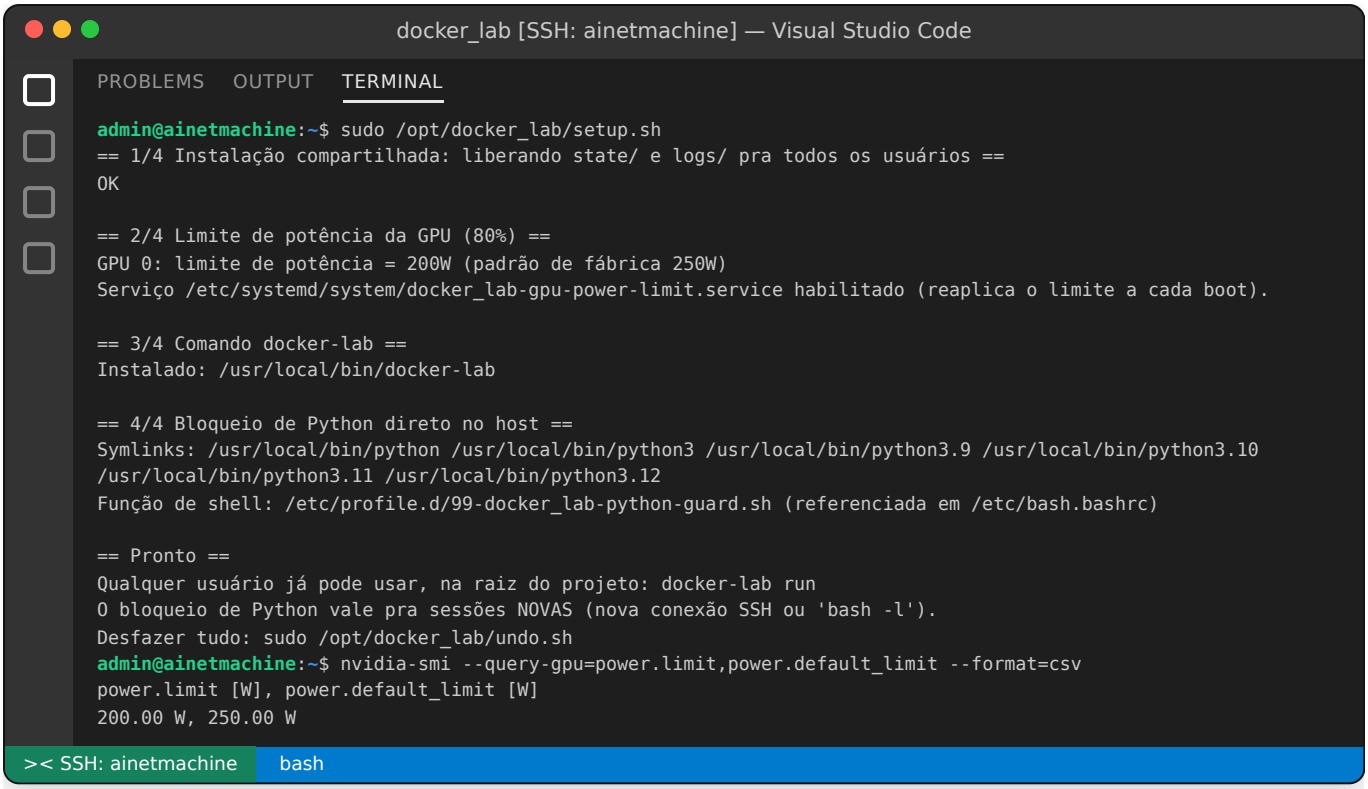


Figura 1 — Instalação pelo terminal do VS Code conectado à máquina do laboratório (Remote-SSH), com a conferência do limite de potência no final.

> Quem estava conectado por SSH durante a instalação só passa a ter o bloqueio completo de Python depois de reconectar. **Para desfazer tudo:** `sudo /opt/docker_lab/undo.sh` . Ele remove o bloqueio, volta a GPU à potência de fábrica, remove o comando `docker-lab` e devolve as permissões originais. Histórico e logs são preservados.

> **E-mails da fila:** o setup cria `/etc/docker_lab/email.env` (só o root lê). Preencha `DOCKERLAB_EMAIL` (uma conta Gmail) e `DOCKERLAB_EMAIL_SENHA_APP` (uma *senha de app* dessa conta) e rode `sudo systemctl restart docker_lab-scheduler` . Sem isso, a fila funciona normalmente, só não envia e-mails.

3. Executando um experimento (cada aluno)

Passo 1 — Criar o `job.env` na raiz do projeto

```
cp /opt/docker_lab/job.env.example ~/meu_projeto/job.env
```

O `job.env` fica **sempre na raiz do projeto**. A pasta onde ele está é montada dentro do container, e os caminhos `SCRIPT` e `PYTHON_BIN` são relativos a ela.

Variável	Exemplo	Significado
`SCRIPT`	`treino.py`	Script a executar, relativo à raiz do projeto
`PYTHON_BIN`	`venv/bin/python`	Python usado. Normalmente é o venv do próprio projeto; são aceitos venvs de Python 3.9 a 3.12.
`CORES`	`2`	Quantidade de cores. O docker_lab escolhe quais cores livres usar.
`MEMORY_GB`	`4`	RAM máxima do job
`GPU`	`none` ou `0`	Não usar GPU (`none`) ou usar a GPU de índice 0, a primeira coluna do `nvidia-smi`
`GPU_MEMORY_GB`	`2`	VRAM máxima (obrigatório com GPU)
`JOB_NAME`	`{USERNAME}_treino`	Nome do job. Mantenha o `{USERNAME}_` , que é trocado pelo seu usuário automaticamente.

> O `nvidia-smi` mostra cerca de 0,3 GB a mais que o `GPU_MEMORY_GB` por job: é a memória fixa que o driver reserva para cada processo (contexto CUDA), e o teto de 80% já deixa folga para ela. Não sabe quanta VRAM o seu script usa? Rode uma vez e veja o pico: `torch.cuda.max_memory_allocated()` no PyTorch, ou `nvidia-smi` durante a execução.

Passo 2 — Executar com `docker-lab run`

Abra o terminal do VS Code na pasta do projeto e rode `docker-lab run job.env`. O experimento entra na **fila** e inicia sozinho, em segundo plano e com o seu usuário, quando houver recursos. A saída mostra se ele já iniciou ou a posição na fila, e o prompt volta na hora. Você pode fechar o VS Code ou desconectar. O `.env` precisa ter `NOTIFY_EMAIL` (avisos por e-mail) e `ESTIMATED_RUNTIME_HOURS` (tempo estimado, em horas).

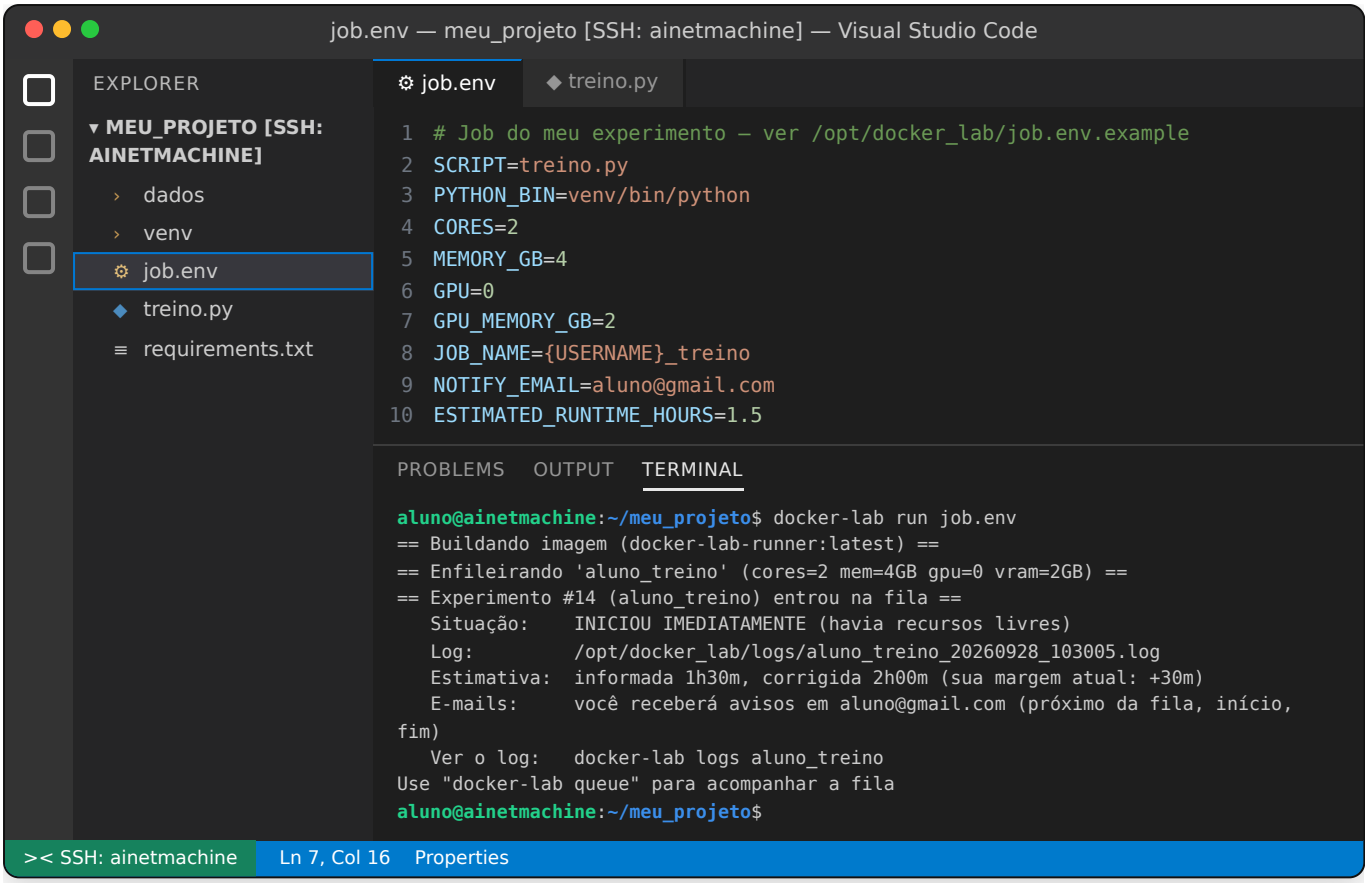


Figura 2 — `job.env` aberto no editor e o job disparado pelo terminal integrado. Com vários arquivos (por exemplo `job_1.env`, `job_2.env`), use `docker-lab run job_1.env`.

Passo 3 — Acompanhar: `docker-lab status` e `docker-lab logs`

`docker-lab status` mostra o que cada recurso tem em uso, o que ainda pode ser reservado dentro do teto da máquina e quais jobs estão ativos. `docker-lab logs job.env` lista todas as execuções daquele `.env` (cada execução tem o próprio log, com data e hora no nome) pergunta qual delas abrir (`Enter` abre a mais recente) e mostra as **últimas 50 linhas**, junto com o caminho do log completo. Para ver outra execução, passe o número dela: `docker-lab logs job.env 1`. No lugar do `.env`, também dá para usar o nome do job que aparece no `docker-lab history` (ex.: `docker-lab logs aluno_treino`), o que permite ver o log de qualquer execução da lista.

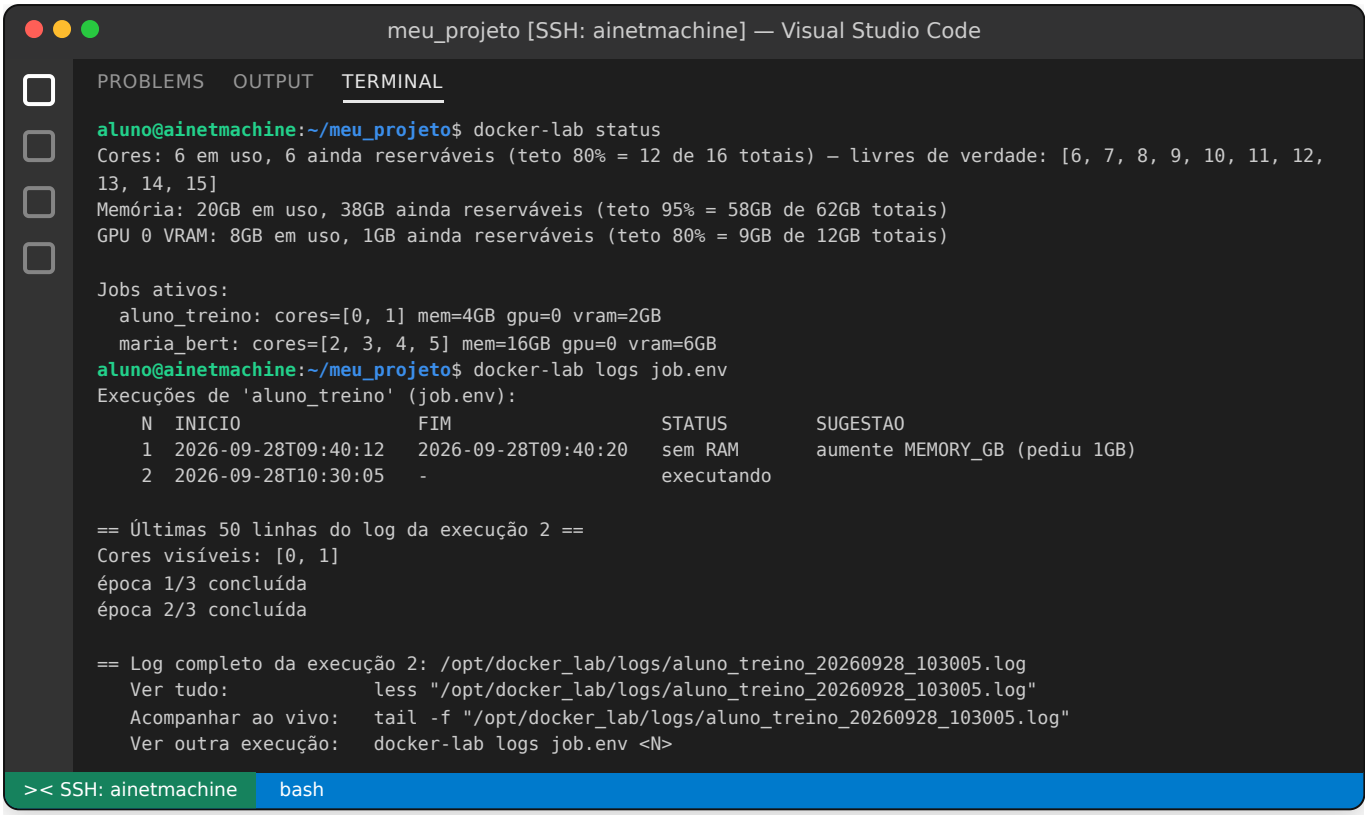


Figura 3 — Dois alunos dividindo a mesma GPU (TITAN V, 12 GB): cada um tem a sua fatia de VRAM, e a soma respeita o teto de 9 GB. O `docker-lab logs` mostra a primeira tentativa, que faltou RAM, e o fim do log da execução atual.

> O job roda com o **seu usuário**: tudo o que o script gravar no projeto (resultados, modelos, logs) é seu, e você pode apagar ou editar sem `sudo`. Só instalar e desinstalar o docker_lab exige `sudo`.

Passo 4 — Parar um job e ver o histórico

`docker-lab stop` lista os jobs ativos num menu numerado. Parar um job **encerra** a execução, que não pode ser retomada depois. A reserva é liberada sozinha. `docker-lab history` mostra todas as execuções com início, fim e status: `executando`, `executou`, `erro`, `interrompido`, `sem RAM` ou `sem VRAM`. Nos dois últimos, a coluna `SUGESTAO` diz o que ajustar no `.env`.

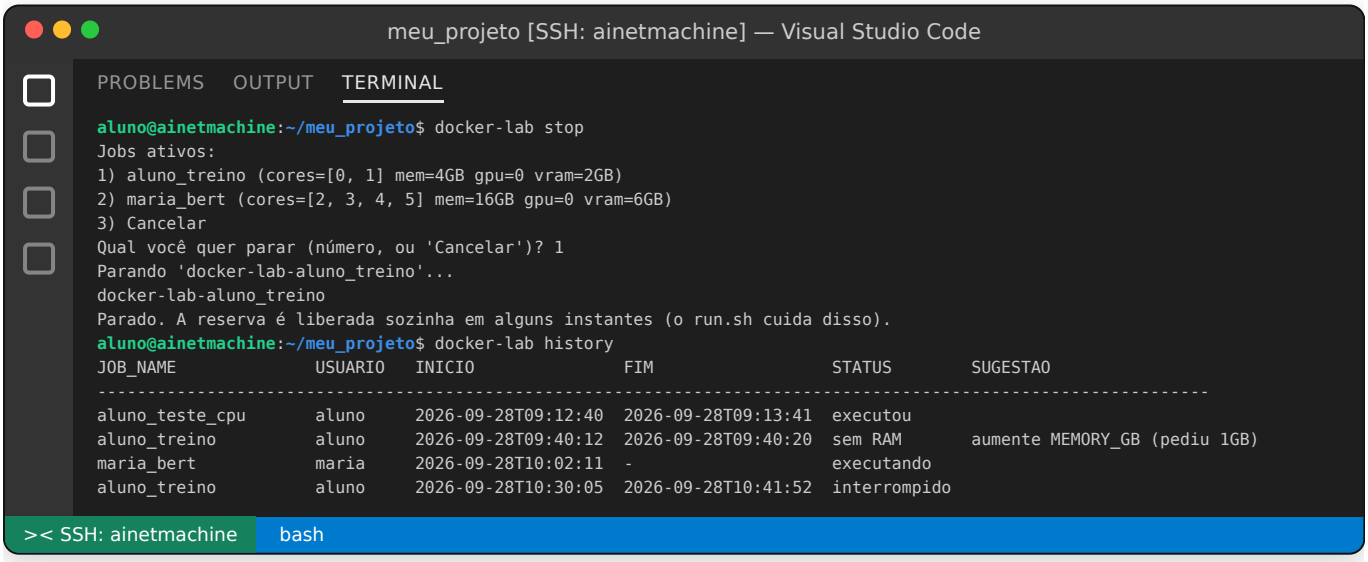


Figura 4 — Parando um job pelo menu e conferindo o histórico. Qualquer usuário pode parar qualquer job, se precisar.

4. O que o docker_lab impede

Rodar Python direto, fora do container

Com o bloqueio ativo, `python3 script.py` no terminal mostra instruções em vez de executar, inclusive com o venv ativado. A sessão do terminal continua normal.

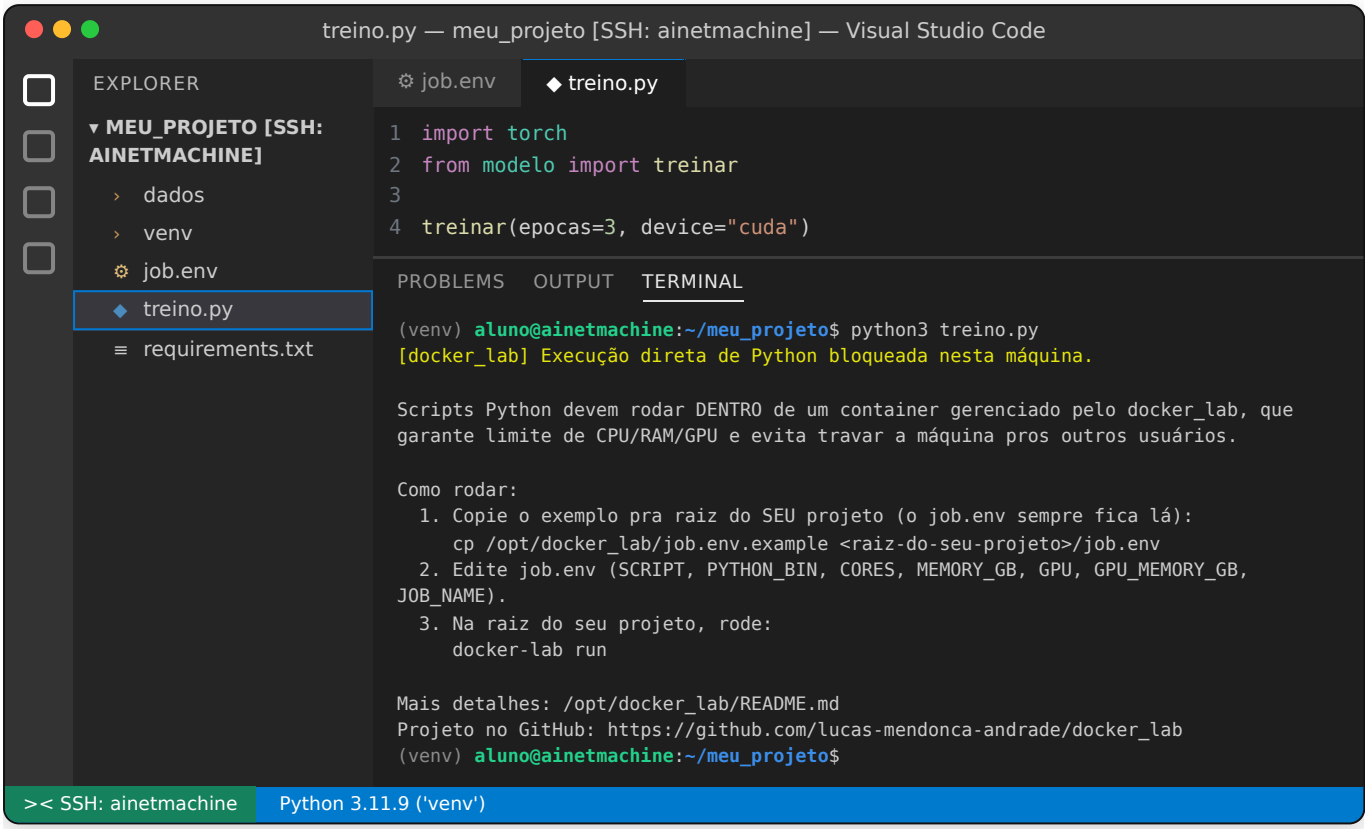


Figura 5 — Tentativa de rodar o script direto, mesmo com o venv ativado: aparece o aviso com o caminho correto.

> O bloqueio é uma **barreira contra esquecimento**, não uma trava de segurança. Caminhos absolutos (`/usr/bin/python3.11 x.py`), `./script.py` e scripts `.sh` não interativos passam sem aviso. Use sempre o `docker-lab run`.

Passar do teto da máquina: fila

Se o pedido de um job, somado ao que já está reservado, passar do teto de qualquer recurso, ele **espera na fila** e inicia sozinho quando houver recursos. A ordem é de chegada, mas um job curto que cabe agora pode passar na frente, se a estimativa corrigida dele terminar antes de liberarem os recursos do primeiro da fila. Acompanhe com `docker-lab queue`. Só é recusado na hora o pedido que **nunca** caberia na máquina, por exemplo mais cores que o teto total.

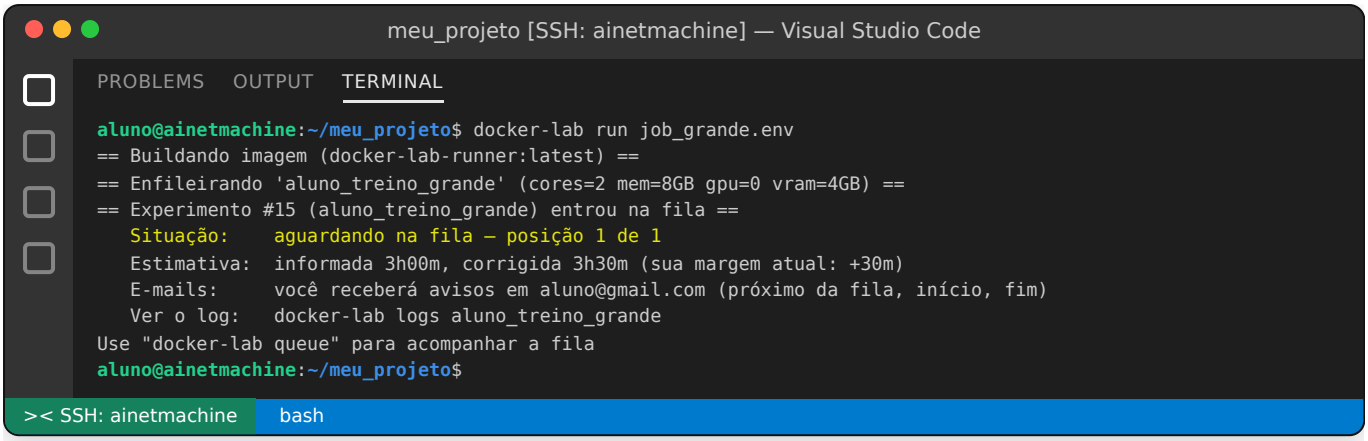


Figura 6 — Pedido de VRAM acima do que está livre agora (a GPU já tem 8 GB reservados, e o teto é 9 GB): o job espera na fila e inicia sozinho quando houver VRAM.

Passar do limite do próprio job

Situação	O que acontece	O que fazer
O script usa mais RAM que `MEMORY_GB`	O container é encerrado. O histórico mostra `sem RAM`, e o fim do log avisa o motivo.	Aumentar `MEMORY_GB`
O script usa mais VRAM que `GPU_MEMORY_GB`	O script recebe `CUDA out of memory`, e só esse job é afetado. Se ele terminar por isso, o histórico mostra `sem VRAM`.	Aumentar `GPU_MEMORY_GB` ou reduzir o batch size
O script pede mais cores (ex. `n_jobs=-1`)	Ele só enxerga os cores reservados e roda com eles.	Nada: é o esperado
`JOB_NAME` já em uso por um job ativo	A execução é recusada.	Usar outro nome no `job.env`

5. Resumo dos comandos

Administrador (uma vez)

```
sudo git clone https://github.com/lucas-mendonca-andrade/docker_lab.git /opt/docker_lab
sudo /opt/docker_lab/setup.sh
# desfazer tudo:
sudo /opt/docker_lab/undo.sh
sudo nano /etc/docker_lab/email.env
```

Aluno (na raiz do projeto)

```
cp /opt/docker_lab/job.env.example job.env
docker-lab run job.env
docker-lab queue
docker-lab status
docker-lab logs job.env
docker-lab stop
docker-lab history
docker-lab corrigir-permissoes
```

Código-fonte, README e atualizações: https://github.com/lucas-mendonca-andrade/docker_lab